

OpenMP Tutorial

@yukim0359

2025.7.10

目次

1. OpenMP とは？
2. 代表的な構文
3. for 文のスケジューリング
4. for 文以外の並列化方法
 - section 構文
 - task 構文
5. 変数の属性
6. スレッドの数に関する小考察
7. Appendix

1. OpenMP とは？
2. 代表的な構文
3. for 文のスケジューリング
4. for 文以外の並列化方法
 - section 構文
 - task 構文
5. 変数の属性
6. スレッドの数に関する小考察
7. Appendix

- **OpenMP** とは，共有メモリ型並列計算機用にプログラムを並列化するための API のこと
 - MP は Multi-Processing を意味する
- C, C++, Fortran から利用可能
- `#pragma omp parallel` などの**コメント形式の指示文**で並列処理を簡単に記述
- 既存の逐次コードに少しずつ並列化を追加できる
- 場合により逐次実行と同じ結果が得られることがあり，デバッグは比較的困難

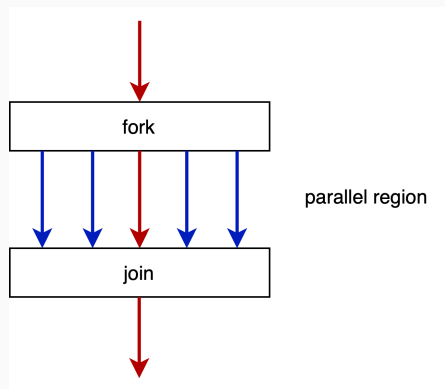
- OpenMP はノード内の並列化（**スレッド間の並列化**）を担う
- **ノード間の並列化**を担当する API として **MPI** などがある
- MPI と違い、データ分散の処理の手間がない分実装は簡単
 - ただし性能は出しにくかったりもする
 - ただスレッドの数を増やせばよいというわけでもない（後述）

- OpenMP 4.0 以降，GPU などのアクセラレータ向けの機能が追加された
- `target` 指示文を使うことで GPU での実行が可能
- 似たようなアクセラレータ向けの指示文 API として **OpenACC** がある
 - ACC は Accelerator を意味する
 - OpenMP と異なり，アクセラレータ向けに最適化された設計
 - 複雑な CUDA コードをほぼ書かずに GPU での実行が可能
- ここでは詳細は触れないが，OpenMP for GPU の単純なプログラム例は Appendix を参照

- 最も基本的な構文
- 以下のように記述することで, `pragma omp parallel` 内部の文を各スレッドが実行する
- 同じ文がスレッドの数だけ実行される点に注意
- ブロックの終わりで終了を待ち合わせる (**fork-join モデル**)

```
1 #pragma omp parallel
2 {
3     // parallel region
4 }
```

fork-join モデル



- 並列領域に入ると，複数のスレッドに分岐 (**fork**)
- 各スレッドが並列に処理を実行
- 並列領域の終わりで同期 (**join**)

- pragma はコンパイラに対する指示文（**ディレクティブ**）で、コンパイラへ指示を与えるために用いる
- C/C++では **#pragma** で始まる行として記述される
- OpenMP では `#pragma omp ...` の形式で並列化の指示をコンパイラに与える
- プラグマの例としては、他にも `#pragma once`（ヘッダファイルの多重インクルード防止）など

- hello.c を実行してみる
 - make でコンパイル可能になっている
 - OpenMP を gcc で利用するためにはコンパイル時に `-fopenmp` をつける
 - mac では gcc を用いると裏で clang が呼び出されてしまい、話が面倒
 - `brew install gcc` で gcc をインストールし、Makefile を `CC := gcc-<ver>` に書き換えるのがおそらく楽
- 実行時は `$ OMP_NUM_THREADS=4 ./hello` のようにすることで実行するスレッド数を指定できる
 - あるいは `$ export OMP_NUM_THREADS=4` として環境変数を設定してもよい
 - `OMP_NUM_THREADS` の指定を省略すると論理コアの数になるはず
 - `omp_set_num_threads(num_threads)` によりプログラム側でも指定できる

- `omp_get_num_threads()` で全体のスレッド数を取得できる
- `omp_get_thread_num()` で自分のスレッド番号を取得できる

```
1 printf("Before parallel region\n");
2 #pragma omp parallel
3 {
4     int tid = omp_get_thread_num();
5     int nthreads = omp_get_num_threads();
6     printf("Hello, World! I'm thread %d of %d\n", tid, nthreads);
7 }
8 printf("After parallel region\n");
```

- 実行結果の例：

```
1 $ OMP_NUM_THREADS=4 ./hello
2 Before parallel region
3 Hello, World! I'm thread 1 of 4
4 Hello, World! I'm thread 2 of 4
5 Hello, World! I'm thread 0 of 4
6 Hello, World! I'm thread 3 of 4
7 After parallel region
```

- たしかに各スレッドが parallel 内部の文を実行している
- 順番は実行のたびに入れ替わる

- 実用上は、同じ処理を並列化するのではなく異なる処理をスレッド間で分散させたい (**Work Sharing 構文**)
 - **for 構文**：`#pragma omp parallel for`
ループのイテレーションを分割
 - **sections 構文**：`#pragma omp parallel sections`
複数の独立処理ブロックを並列実行
 - **task 構文**：`#pragma omp task`
任意の処理を動的にタスク化して柔軟に分散

1. OpenMP とは？
2. 代表的な構文
3. for 文のスケジューリング
4. for 文以外の並列化方法
 - section 構文
 - task 構文
5. 変数の属性
6. スレッドの数に関する小考察
7. Appendix

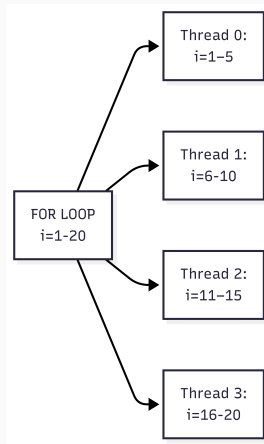
for 構文

- 以下のように書くことで, for 文の各イテレーションがスレッドに分散される

```
1 #pragma omp parallel
2 {
3     #pragma omp for
4     for (int i=0; i<N; i++) {
5         a[i] = i;
6     }
7 }
```

- 以下のように書いても同じ

```
1 #pragma omp parallel for
2 for (int i=0; i<N; i++) {
3     a[i] = i;
4 }
```



parallel_for.c の実行

- parallel_for.c を実行してみる
- 実行結果の例：

```
1 $ OMP_NUM_THREADS=4 ./parallel_for
2 Loop 2 is being processed by thread 1
3 Loop 3 is being processed by thread 1
4 Loop 4 is being processed by thread 2
5 Loop 5 is being processed by thread 2
6 Loop 0 is being processed by thread 0
7 Loop 1 is being processed by thread 0
8 Loop 6 is being processed by thread 3
9 Loop 7 is being processed by thread 3
```

- 若い番号から (ループの個数)/(スレッドの個数) ごと各スレッドにループが割り当てられる
 - この割り当ての方法は変更可能 (後述)

- for 文の並列化においては注意すべき点がある
 1. 共有変数とプライベート変数について
 2. 並列化できない例
 3. データ競合

注意点①：共有変数とプライベート変数について

- 変数にはスレッドごとに共有されるもの（共有変数）とスレッドごとに独立に確保されるもの（プライベート変数）がある
 - これを変数の属性という
- `pragma omp parallel for` の直後のループ変数以外はすべて共有変数となる (!)
- つまり、以下のようなプログラムは正しく動作しない
 - `j` が共有変数
 - `j = i` と `a[j] = i` の間に `j` が他スレッドにより書き換えられる可能性

```
1 #pragma omp parallel for
2 for (i=0; i<N; i++) {
3     j = i;
4     a[j] = i;
5 }
```

注意点①：共有変数とプライベート変数について

- 実際におかしなことが起こるのを確認する
- attribution_err.c の実行結果

```
1 $ OMP_NUM_THREADS=4 ./attribution_err
2 Sum = 9986
3 $ OMP_NUM_THREADS=4 ./attribution_err
4 Sum = 9989
5 $ OMP_NUM_THREADS=1 ./attribution_err
6 Sum = 10000
```

- 正しい結果は Sum = 10000

注意点①：共有変数とプライベート変数について

- 正しくはこう，private 節で個別にプライベート変数に設定できる

```
1 #pragma omp parallel for private(j)
2 for (i=0; i<N; i++) {
3     j = i;
4     a[j] = 1;
5 }
```

- あるいは default(private) とすればデフォルトがプライベート変数になる

注意点①：共有変数とプライベート変数について

- あるいはこれでもいい
 - parallel 節内部で宣言された変数は private になるはず

```
1 #pragma omp parallel for
2 for (i=0; i<N; i++) {
3     int j = i;
4     a[j] = 1;
5 }
```

- 変数の属性について、もう少し細かい話は後述

注意点②：並列化できない例

- すべての for 文が並列化可能なわけではない
- i を 1 から N まで順番に回さないと逐次実行と結果が異なるようなプログラム（流れ依存）に対しては、（アルゴリズムの変更等を試みない限り）厳しい
- 例 1：

```
1 for (i=0; i<N; i++) {  
2     a[i] = a[i] + 1;  
3     b[i] = a[i-1] + a[i+1];  
4 }
```

- $a[i-1]$ が更新されていない可能性がある

注意点②：並列化できない例

- 例2：

```
1 for (i=0; i<N; i++) {  
2     a[i] = a[b[i]];  
3 }
```

- b[i] の内容により並列化が可能かどうか決まる

注意点③：データ競合

- （オペレーティングシステムの授業で学んだように）並列処理においてはデータ競合への対処が重要
- たとえば以下のようなプログラムは正しく動作しない

```
1 #pragma omp parallel for
2 for (int i = 0; i < N; i++) {
3     for (int j = 0; j < N; j++) {
4         A[j] += 2.0 * B[i];
5     }
6 }
```

- A[j] に対する読み込み→書き込みの間に他スレッドが書き込みを行う可能性がある
- OpenMP ではデータ競合を避ける方法がいくつかある

データ競合の回避①：lock

- OpenMP では lock 機構が用意されている
- 先ほどの例だと以下のよう

```
1  omp_lock_t lock;  
2  omp_init_lock(&lock);  
3  #pragma omp parallel for  
4  for (int i = 0; i < N; i++) {  
5      for (int j = 0; j < N; j++) {  
6          omp_set_lock(&lock);  
7          A[j] += 2.0 * B[i];  
8          omp_unset_lock(&lock);  
9      }  
10 }
```

データ競合の回避②：critical 補助指示文

- `pragma omp critical` 内部の文は、同時には1つ以下のスレッドしか実行しない
- `lock` より自由度は低いが、実装は単純
- オプション (**name**) をつけることで複数のクリティカルセクションを作ることができる
 - 同じ `name` を持つクリティカルセクション同士は排他制御されるが、異なる `name` なら同時に実行できる
 - `name` は省略可能
- 先ほどの例だと以下のように

```
1 #pragma omp parallel for
2 for (int i = 0; i < N; i++) {
3     for (int j = 0; j < N; j++) {
4         #pragma omp critical
5         {
6             A[j] += 2.0 * B[i];
7         }
8     }
9 }
```

データ競合の回避③：atomic 補助指示文

- **単純な式一行のみ**（`x+=値` や `x^=値` など）であれば、`pragma omp atomic` を用いると速い
- ハードウェア命令により値を更新する
- 先ほどの例だと以下のよう

```
1 #pragma omp parallel for
2 for (int i = 0; i < N; i++) {
3     for (int j = 0; j < N; j++) {
4         #pragma omp atomic
5         A[j] += 2.0 * B[i];
6     }
7 }
```

lock, critical, atomic の比較

- これらの実行時間を比較してみる
- 実行時間の計測は `omp_get_wtime()` により可能
- `lock.c`, `critical.c`, `atomic.c` を実行
- 実行結果の例：

```
1 $ ./lock
2 Elapsed time: 4.004458 seconds
3 $ ./critical
4 Elapsed time: 4.324831 seconds
5 $ ./atomic
6 Elapsed time: 0.070357 seconds
```

lock, critical, atomic の比較

- だが、そもそもこの比較はおかしい
- 上のプログラムでは1つのlock, 1つのcriticalセクションを利用したが、これによって不必要な待機時間が発生しており、逐次実行とほぼ同じになっている
- 本来避けるべきは「**同じ A[j] へのアクセス**」なので、A[j] ごとに1つのlockを用意すればよい
- 以下のように：

```
1  omp_lock_t  locks[N];
2  for (int i = 0; i < N; i++) omp_init_lock(&locks[i]);
3  #pragma omp parallel for
4  for (int i = 0; i < N; i++) {
5      for (int j = 0; j < N; j++) {
6          omp_set_lock(&locks[j]);
7          A[j] += 2.0 * B[i];
8          omp_unset_lock(&locks[j]);
9      }
10 }
```

- critical の name は (おそらく) 動的に名前をつけることができないため, 上のような実装は難しく, 省略
- lock2.c を実行
- 実行結果の例:

```
1 $ ./atomic
2 Elapsed time: 0.070357 seconds
3 $ ./lock2
4 Elapsed time: 0.161204 seconds
```

- 簡単に言えば、複数のスレッドで総和をとるときなどに使うことができる
- 単純な例：ベクトルの内積計算

```
1 // Implementation using the atomic directive
2 #pragma omp parallel for
3 for (int i=0; i<N; i++) {
4     #pragma omp atomic
5     sum += A[i] * B[i];
6 }
7
8 // Implementation using the reduction clause
9 #pragma omp parallel for reduction(+:sum)
10 for (int i=0; i<N; i++) {
11     sum += A[i] * B[i];
12 }
```

- 「+」以外にもいくつかの演算、組み込み手続きが利用可能 (max, min など)
- ユーザ定義の演算にも対応している

reduction の実装

- まず各スレッドが自分のローカルな変数で計算を行う
- 最後にそれらを結合する
- 以下のようなイメージ

```
1 #pragma omp parallel
2 {
3     int tid = omp_get_thread_num();
4     #pragma omp for
5     for (int i=0; i<N; i++) {
6         sum_local[tid] += A[i] * B[i];    // Each thread updates only its own
        sum_local
7     }
8 }
9
10 for(int i=0; i<nthreads; i++){
11     sum += sum_local[i];    // Aggregate partial sums from all threads into the
        final sum
12 }
```


- 「和を取る操作」に対して, atomic, reduction, private-array の3つの実装を比較 (private-array とは, 前ページのような操作を自前で実装したもの)
- `sum_compare.c` を実行
- 実行結果の例:

```
1 Elements: 100000, Threads: 4
2 1) reduction      : sum = 200000.000000   time = 0.000297 s
3 2) atomic         : sum = 200000.000000   time = 0.002980 s
4 3) private-array  : sum = 200000.000000   time = 0.000370 s
```

1. OpenMP とは？
2. 代表的な構文
3. for 文のスケジューリング
4. for 文以外の並列化方法
 - section 構文
 - task 構文
5. 変数の属性
6. スレッドの数に関する小考察
7. Appendix

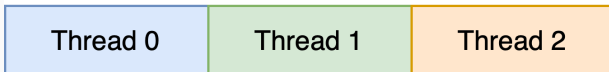
- 当然，高性能化のためにはスケジューリングが重要
- 並列化したとしても，負荷分散が不均衡だと性能が出ない
- 仕事の最も多いスレッドが終わるまで待たないといけない

スケジューリングの種類

- for ループのスケジューリングの種類には以下の3通りがある
 1. **static**: スレッドごとに chunk 個のイテレーションを静的に割り当てる
 2. **dynamic**: スレッドごとに chunk 個のイテレーションを動的に割り当てる, 終わったスレッドから次の chunk を取りに行く
 3. **guided**: スレッドごとに割り当てるイテレーションの個数を徐々に小さくする
- `#pragma omp parallel for` **`schedule(dynamic, chunk)`** のように指定できる (省略した場合のデフォルトは `static`)

- `schedule(static, chunk)`
 - ループをチャンクサイズ `chunk` で分割し、スレッド 0 番からラウンドロビン方式で静的に割り当てる
 - `schedule` 補助指定文を省略したときのデフォルトは `static` で、チャンクサイズは (ループの回数)/(スレッドの回数) である (参考: 02_syntax/parallel_for.c)

$\text{chunk} = (\text{iter_num}) / (\text{threads_num})$



$\text{chunk} = 1$



- `schedule(dynamic, chunk)`
 - ループをチャンクサイズ `chunk` で分割し、処理が終わったスレッドに順々に次の処理を割り当てる
 - 実行時にならないとループ内部の計算量が明らかにならない場合に有効
 - `chunk` を省略すると 1 になる



- `schedule(guided, chunk)`
 - ループをチャンクサイズで分割し，徐々にチャンクサイズを小さくしながら，処理が終わったスレッドに順々に次の処理を割り当てる
 - `chunk` はチャンクサイズの最小単位
 - チャンクサイズに 1 より大きい k を指定した場合，チャンクサイズは指数的に k まで小さくなるが，最後のチャンクは k より小さくなる場合がある
 - `chunk` を省略すると 1 になる



各スケジューリングの特徴

- dynamic, guided のチャンクサイズは性能に影響する
 - チャンクサイズが小さすぎると負荷バランスは良くなるが、処理待ちのオーバーヘッドは大きくなる
 - チャンクサイズが大きすぎると負荷バランスは悪くなるが、処理待ちのオーバーヘッドは小さくなる
 - チャンクサイズのチューニングは重要
- 割り当てのオーバーヘッド
 - dynamic, guided は実行時スケジューリングのため、static よりもオーバーヘッドが大きい
 - **事前に負荷分散が均衡となるスケジューリングを組むことができれば**, static スケジューリングを用いることでさらに高速化できる可能性がある

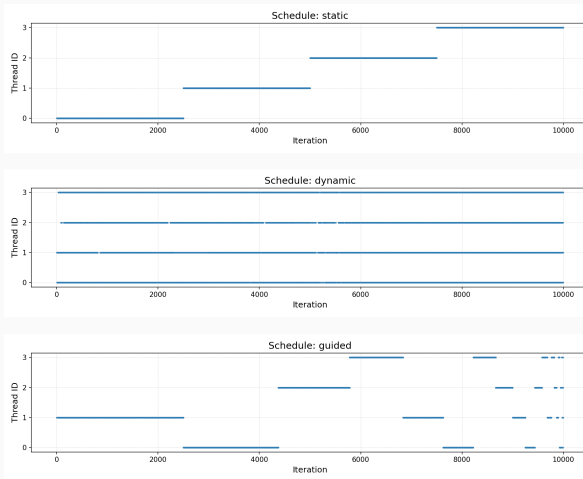
スケジューリングの比較

- 以下のような for ループの外側のみを OpenMP で並列化する（ループの後半ほど計算量が多い）
- スケジューリングとして上記の 3 パターンを試し，実行時間を比較する
- どのループをどのスレッドが担当したかの記録も取っておく

```
1 for (int i=0; i<N; i++) {  
2     for (int j=0; j<i; j++) {  
3         A[i]++;  
4     }  
5 }
```

スケジューリングの比較 計測結果

- static.c, dynamic.c, guided.c を実行（スレッド数は4）



- 実行結果の例：

```
1 === OpenMP threads: 4 ===  
2 static : 0.061116 s  
3 dynamic : 0.052824 s  
4 guided : 0.043387 s
```

- 計測時間のブレは大きかったため、厳密性には欠けるが：
- static が最も遅い
 - 後半を担当したスレッドほど長く時間がかかるために負荷分散が均衡にならないと考えられる
- dynamic より guided の方が速い
 - 序盤は大雑把に割り当てる分、割り当てのオーバーヘッドが dynamic より小さいと考えられる

static のチャンクサイズを変える

- チャンクサイズを 1 とすればより均等に分割される（もともとのチャンクサイズは (ループの個数)/(スレッドの個数) であったことに注意)
- static の chunk を 1 として実行

```
1 === OpenMP threads: 4 ===  
2 static : 0.045067 s  
3 dynamic : 0.052824 s  
4 guided : 0.043387 s
```

- 静的割り当てなぶん guided より速くなるかと思ったが、明確に速くなるわけでもなかった
 - キャッシュヒット率の違い？
guided は序盤は連続アクセス，static は 4 個飛びでバラバラ
 - そもそも割り当てのオーバーヘッドがどのオーダーなのかわかっていない

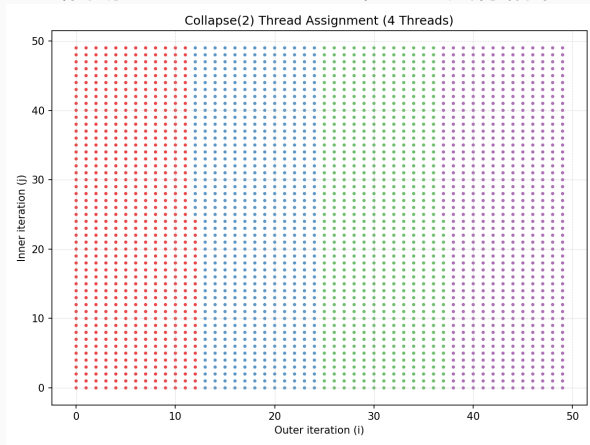
- ネストされたループを並列化する際は collapse 補助指示文を用いることができる

```
1 #pragma omp parallel for collapse(2)
2 for (int i=0; i<N; i++) {
3     for (int j=0; j<N; j++) {
4         A[i] += B[j];
5     }
6 }
```

- for 文の間に何か操作を挟むことはできない（純粋な二重ループでないといけない）
- collapse(n) で n 層のループを**大きな一つのループとみなして**並列化できる
- schedule 補助指示文と組み合わせることもできる

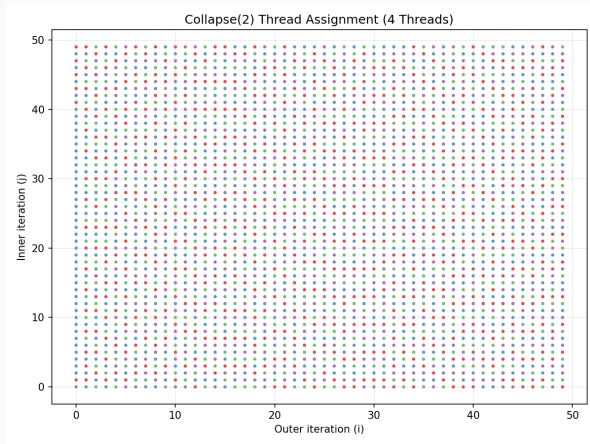
collapse の挙動を調べてみる

- collapse.c を実行
- スレッドごとに色分けしてどのループを担当しているか可視化
- schedule の指定なし（自動的に static になる）での実行結果：



collapse の挙動を調べてみる

- `schedule(dynamic, 1)` での実行結果：



- 二重ループを展開することで、分散すべきイテレーションの個数が増える
 - 上記のような CPU 上のプログラムではあまり嬉しさはない？
- 役に立つ例としては、GPU 上での for 文の並列化などが考えられる
- 詳細は 07_appendix/matmul_gpu.c を参照

1. OpenMP とは？
2. 代表的な構文
3. for 文のスケジューリング
4. for 文以外の並列化方法
 - section 構文
 - task 構文
5. 変数の属性
6. スレッドの数に関する小考察
7. Appendix

ループ以外の並列化①：section 構文

- for 構文よりもう少し自由度の高い (?) 並列化方法
- 各 section がスレッドに割り当てられる

```
1 #pragma omp parallel sections
2 {
3 #pragma omp section
4 {
5     work1();
6 }
7 #pragma omp section
8 {
9     work2();
10 }
11 }
```

- 詳細は略

1. OpenMP とは？
2. 代表的な構文
3. for 文のスケジューリング
4. for 文以外の並列化方法
 - section 構文
 - task 構文**
5. 変数の属性
6. スレッドの数に関する小考察
7. Appendix

その前に：single 構文

- single 構文の内部は，1つのスレッドのみが実行する
- どのスレッドが実行するかは不定
- nowait 補助指定文を用いない限り，**single 終了後に同期が入る**
- 詳細は single.c も参照

```
1 #pragma omp parallel
2 {
3     #pragma omp single
4     {
5         work();
6     }
7 }
```

その前に：master 構文

- single 構文と似ている
- ただし、master 構文は**必ずマスタスレッドが実行する**
- **終了後に同期が入らない**
- 詳細は master.c も参照

```
1 #pragma omp parallel
2 {
3     #pragma omp master
4     {
5         work();
6     }
7 }
```

ループ以外の並列化②：task 構文

- 動的・不規則に生成される仕事の並列化に向く
- `task` でタスクを生成
- `taskwait` でタスクの完了を待機

ループ以外の並列化②：task 構文

- 例：フィボナッチ数列の計算

```
1  int fib(int n) {
2      if (n<2) return n;
3      else{
4          #pragma omp task shared(x) firstprivate(n)
5          x = fib(n - 1);
6          #pragma omp task shared(y) firstprivate(n)
7          y = fib(n - 2);
8          #pragma omp taskwait
9          return x + y;
10     }
11 }
12
13 #pragma omp parallel
14 {
15     #pragma omp master
16     printf("Fibonacci of %d is %d\n", 10, fib(10));
17 }
```

fib.c の実行

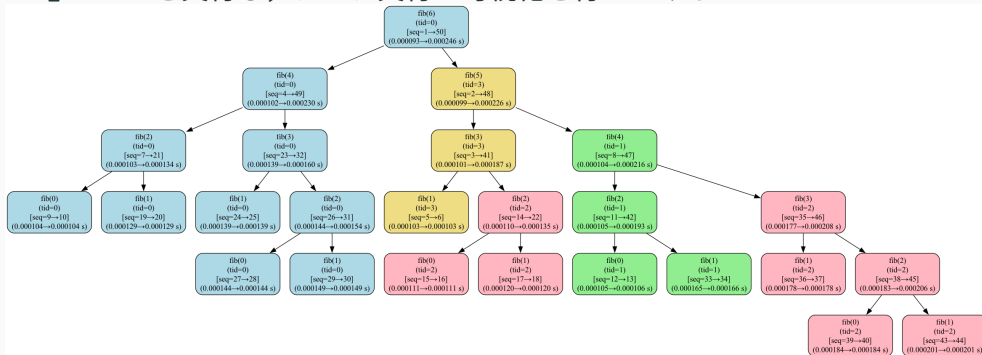
- fib.c を実行
- 実行結果の例：

```
1 $ ./fib
2 serial version
3 Fibonacci of 50 is 12586269025
4 Elapsed time: 51.424394 seconds
5
6 $ OMP_NUM_THREADS=4 ./fib
7 OpenMP version
8 threads: 4
9 Fibonacci of 50 is 12586269025
10 Elapsed time: 23.339175 seconds
```

- タスクの作成や管理にはオーバーヘッドが生じる
 - $n = 50$ だとおよそ 2^{50} 個のタスク!?
 - n が小さいときは逐次実行とすることでこれを避ける

タスク実行の可視化

- fib_tree.c を実行し、タスク実行の可視化を行ってみる



- [GIF へのリンク](#)

1. OpenMP とは？
2. 代表的な構文
3. for 文のスケジューリング
4. for 文以外の並列化方法
 - section 構文
 - task 構文
5. 変数の属性
6. スレッドの数に関する小考察
7. Appendix

- **private** : 各スレッドに独立した領域を割り当て、**並列領域開始時の値は不定**
- **shared** : 全スレッドで同じ変数を参照、並列領域内で競合すると未定義動作になるため要同期
- **firstprivate** : private と同じだが、**並列領域開始時に親スレッドの値で初期化**
- **lastprivate** : 並列領域終了後、最後に更新されたスレッドの値を親スレッドに反映
- 詳細は `attribution.c` を参照

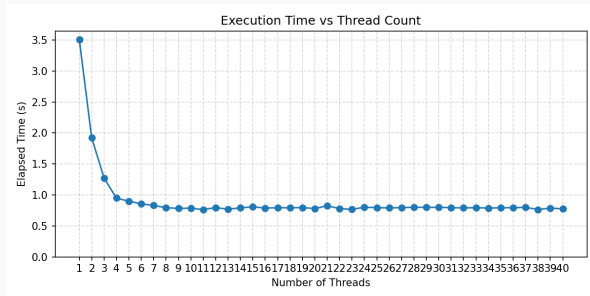
1. OpenMP とは？
2. 代表的な構文
3. for 文のスケジューリング
4. for 文以外の並列化方法
 - section 構文
 - task 構文
5. 変数の属性
6. スレッドの数に関する小考察
7. Appendix

最適なスレッド数は？

- スレッドを増やすほど性能が向上するわけではない
- **論理コアの数**までしかスレッドは同時に実行できない
- 利用可能な論理コア数を知るためのコマンド
 - Linux : `nproc`
 - Mac : `sysctl -n hw.logicalcpu`
- あるいは `max_threads.c` を実行
 - `omp_get_max_threads()` は利用可能な最大のスレッド数を返す
- 前田の Mac では論理コア数は 8

スレッド数と実行時間の関係①

- 行列積を並列化して計算するプログラム `measure_threads_1.c` をスレッド数を変えながら実行
- 実行結果の例：



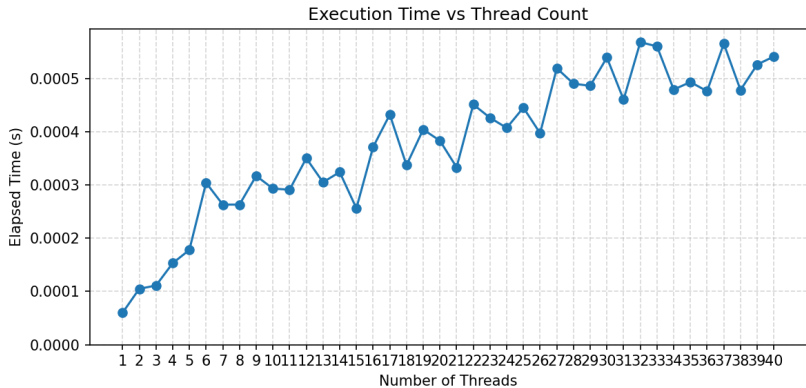
- 8 スレッド程度で性能は頭打ち
 - たしかに論理コア数あたりが上限

スレッド数と実行時間の関係②

- 内容によってはスレッド数を増やすことで性能が悪化することもある
 - たとえば排他制御を含むプログラム
 - lock を獲得した時点で OS によって実行が中断されると（**プリエンプション**），再び実行状態になるまでの時間がタイムロスとなる
- 単純にカウンタをインクリメントするプログラム `measure_threads_2.c` をスレッド数を変えながら実行

スレッド数と実行時間の関係②

- 実行結果の例：



- スレッドを増やすほど性能は悪化
- 並列化により性能が良くなるかはプログラムの構造に依存する

1. OpenMP とは？
2. 代表的な構文
3. for 文のスケジューリング
4. for 文以外の並列化方法
 - section 構文
 - task 構文
5. 変数の属性
6. スレッドの数に関する小考察
7. Appendix

1. OpenMP における SIMD 機能
2. プログラム例
3. 各種ディレクティブの実装

- SIMD を用いると、同じ命令を複数のデータに対して並列に実行できる
- OpenMP では `#pragma omp simd` で SIMD 機能を使用できる

```
1 #pragma omp simd
2 for (int i = 0; i < N; i++) {
3     b[i] = 2 * a[i];
4 }
```

参考：コンパイルと最適化レポート

- 以下のようにコンパイルすることで、ベクトル化に関する最適化レポートを `vec_report.txt` に出力できる
- `-O3` は最適化レベルを指す

```
1 $ gcc-15 -O3 -fopt-info-vec-optimized=vec_report.txt -fopenmp simd.c -o simd
```

- 出力例：

```
1 simd.c:30:21: optimized: loop vectorized using 16 byte vectors
2 simd.c:30:21: optimized: loop vectorized using 8 byte vectors
3 simd.c:18:10: optimized: loop vectorized using 16 byte vectors
```

- たしかに SIMD 化されていることがわかる

参考：アセンブリの生成

- 以下のようにコンパイルすることで、アセンブリを `simd.s` に出力できる

```
1 $ gcc-15 -O1 -fopenmp -S simd.c -o simd.s
```

- 出力例：

```
1 ldr q31, [x7, x1]           // x7 + x1 のアドレスから  
    128bitロード (int型4つ分)  
2 add v31.4s, v31.4s, v31.4s  //  
    v31の4つの32bit整数をそれぞれ2倍 (ベクトル加算)  
3 str q31, [x0, x1]          // x0 + x1 のアドレスに  
    128bitストア (結果を書き込み)
```

- ベクトルレジスタにデータを格納後、4つの整数を一気に2倍している
- Mac の場合、Arm Neon という SIMD 拡張が採用されているようである
- SIMD 命令のために用意されているレジスタ幅が 128bit のため、32bit 整数 4 つや 64bit 小数 2 つを一気に演算できる

SIMD の有無による実行時間の比較

- simd.c を実行
- SIMD ありの場合（最適化レベル -O1 のうえに #pragma omp simd を指定し，SIMD 命令が有効化されていることを確認）

```
1 $ OMP_NUM_THREADS=1 ./simd
2 Dot product (SIMD): 0.161255 sec
3 $ OMP_NUM_THREADS=4 ./simd
4 Dot product (SIMD): 0.151914 sec
```

- SIMD なしの場合（最適化レベル -O1 のうえに #pragma omp simd を指定せず，SIMD 命令が有効化されていないことを確認）

```
1 $ OMP_NUM_THREADS=1 ./simd
2 Dot product (scalar): 0.215824 sec
3 $ OMP_NUM_THREADS=4 ./simd
4 Dot product (scalar): 0.151753 sec
```

- 測定の誤差には留意する
- スレッド数が小さいほど SIMD の効果は高い（が、4 倍になったりはしない）
 - スレッド数が増えるほどメモリアクセスが律速になるため
- なお、自分の環境・このプログラムでは、そもそも最適化レベルを `-O3` にしてコンパイルすると、`#pragma omp simd` を指定しなくても SIMD 命令が有効化された

- モンテカルロ法による円周率の近似計算
 - ランダムに点を打ち，円内の点の割合から円周率を計算
 - 各スレッドで独立に点を生成し，円内の点の数をカウント
 - 最後に全スレッドの結果を合計（reduction）して円周率を計算
- `monte_carlo.c` を参照

- 再帰的なマージソートを OpenMP の task を用いて並列化
 - 配列を 2 分割し，それぞれのソートをタスクとして登録
 - 分割が十分小さくなったら挿入ソートなどで逐次処理
 - 並列タスクの終了を `taskwait` で待機し，最後にマージ
 - 並列 for では困難な「再帰的な分割と並列化」が可能
- `parallel_merge_sort.c` を参照

GPU 実行のプログラム例：行列積

- 行列積の GPU での並列化を OpenMP で実装する
- matmul_gpu.c を参照

```
1 #pragma omp target data map(to:A[0:N][0:N], B[0:N][0:N]) map(from:C[0:N][0:N])
   ]
2 {
3     #pragma omp target teams distribute parallel for collapse(2)
4     for(int i = 0; i < N; i++) {
5         for(int j = 0; j < N; j++) {
6             for(int k = 0; k < N; k++) {
7                 C[i][j] += A[i][k] * B[k][j];
8             }
9         }
10    }
11 }
```

GPU 実行のプログラム例：行列積

- ここでは `collapse(2)` の有無による実行時間を比較してみる (miyabi-g 上で実行)
- 実行結果の例：

```
1 $ ./matmul_gpu
2 Use collapse
3 Test passed!
4 Computation time: 0.386624 seconds
5
6 $ ./matmul_gpu
7 No collapse
8 Test passed!
9 Computation time: 0.702773 seconds
```

- `collapse` によりイテレーション個数が N から N^2 に増え、GPU 上の多数のスレッドに計算が行き渡ると考えられる

- 細部まで目を通せているわけではないので、間違いがあってもご容赦いただきたい
- 以下 gcc における OpenMP ランタイム内部実装 (`libgomp`) に限って話を進める
- 実装は
`https://github.com/gcc-mirror/gcc/tree/releases/gcc-9.2.0/libgomp` に示されている

各種ディレクティブの実装①：スレッドの生成

- parallel ディレクティブは

```
1 void subfunction (void *data)
2 {
3     use data;
4     body;
5 }
6
7 setup data;
8 GOMP_parallel_start (subfunction, &data, num_threads);
9 subfunction (&data);
10 GOMP_parallel_end ();
```

と展開される

- GOMP_parallel_start が gomp_team_start() を呼び出し、OS レベルのスレッドを生成 (Linux, Mac : pthread_create)
- git リポジトリの parallel.c, team.c 参照

各種ディレクティブの実装②：lockの実装

- 自分の調べた範囲では pthread_mutex と同等
- config/posix/mutex.h に以下のように定義されている

```
1 typedef pthread_mutex_t gomp_mutex_t;  
2 static inline void gomp_mutex_init (gomp_mutex_t *mutex)  
3 {  
4     pthread_mutex_init (mutex, NULL);  
5 }  
6 static inline void gomp_mutex_lock (gomp_mutex_t *mutex)  
7 {  
8     pthread_mutex_lock (mutex);  
9 }  
10 // 略
```

- lock.c 参照

各種ディレクティブの実装③：criticalの実装

- 名前なし critical セクションの定義

```
1 void
2 GOMP_critical_start (void)
3 {
4     /* There is an implicit flush on entry to a critical region. */
5     __atomic_thread_fence (MEMMODEL_RELEASE);
6     gomp_mutex_lock (&default_lock);
7 }
8
9 void
10 GOMP_critical_end (void)
11 {
12     gomp_mutex_unlock (&default_lock);
13 }
```

- 結局 mutex で管理していそう
- critical.c 参照

各種ディレクティブの実装④：atomicの実装

- 07_appendix/atomic1.c をアセンブリにすると, atomic 部分は以下ようになる

```
1 ldadd w1, w0, [x0]
```

- arm において ldadd は「メモリ位置 [x0] から値をロード→w0 の値を加算→結果を [x0] にストア→元のメモリ値を w1 に返す」を**不可分**で実行する命令

各種ディレクティブの実装④：atomicの実装

- もう少し複雑だとどうなるか
- 07_appendix/atomic2.c をアセンブリにすると, atomic 部分は以下ようになる

```
1 LFB0:
2     fadd    d0, d0, d0        // bを2倍
3     ldr     x1, [x0]          // *aをx1に読み出し
4 L2:
5     fmov    d31, x1           // x1を浮動小数点レジスタd31に移動
6     mov     x2, x1            // x1をx2にコピー
7     fadd    d31, d0, d31      // d31 = *a + 2*b
8     fmov    x3, d31           // x3に新しい値を格納
9     cas     x2, x3, [x0]      // メモリ内の現値[x0]が
    x2 (旧値) と一致すればx3 (新値) をatomicに書き込み
10                                // いずれの場合も元のメモリ値をx2に戻す
11     cmp     x1, x2            // casの結果, x1==x2なら成功
12     bne     L3                // 失敗なら再試行
13     ret
```

- CAS ループになっている

- http://www.den.t.u-tokyo.ac.jp/ad_prog/parallel/
- https://www.hpci-office.jp/documents/HPC_Programming_Seminar/openmp-part2-20240417.pdf
- <https://theartofhpc.com/pcse/index.html>
- <https://www.openmp.org>
- <https://gcc.gnu.org/onlinedocs/libgomp/>
- <https://developer.arm.com/documentation/dht0002/a/Introducing-NEON?lang=en>
- 片桐 孝洋「並列プログラミング入門：サンプルプログラムで学ぶ OpenMP と OpenACC」
- 東京大学 スパコンプログラミング 講義資料